



Politecnico di Torino

Course:
Optimization and Simulation for Industrial Automation

Final Project

Small Language Model (SLM) System

Queueing Analysis and Scheduling Optimization

Authors:

Ali Vaezi
s336256@studenti.polito.it
Interuniversity Department of Regional
and Urban Studies and Planning (DIST)

Riccardo Givogre
s349452@studenti.polito.it
Department of Control and
Computer Engineering (DAUIN)

Academic Year 2025/2026

January 2026

1 Introduction

With the rapid advancement of artificial intelligence and natural language processing, Small Language Models (SLMs) have become increasingly important in industrial automation applications. These models require efficient computational pipelines that typically involve multiple processing stages across different hardware resources. Optimizing such systems is crucial for minimizing response times and maximizing resource utilization.

This project analyzes a Small Language Model (SLM) system consisting of CPU and GPU resources arranged in a flow-shop configuration. Each prompt must pass through three stages: pre-processing (tokenization) on CPU1, inference on one of two parallel GPUs (GPU1 or GPU2), and post-processing (de-tokenization) on CPU2. We investigate this system under two distinct operational scenarios representing different use cases.

1.1 System Architecture

The SLM system consists of four processing units arranged in a flow-shop configuration:

1. **CPU 1 (Pre-processing):** Performs tokenization of incoming prompts. Service rate $\mu_C = 8$ prompts/min.
2. **GPU 1 and GPU 2 (Inference):** Two parallel GPUs perform the language model inference. Each GPU has service rate $\mu_G = 5$ prompts/min.
3. **CPU 2 (Post-processing):** Performs de-tokenization of model outputs. Service rate $\mu_C = 8$ prompts/min.

The processing times are exponentially distributed with means $1/\mu_C = 0.125$ minutes for CPU stages and $1/\mu_G = 0.2$ minutes for GPU stages. The GPU stage is the bottleneck of the system since it has a slower service rate than the CPU stages.

1.2 Project Objectives

The main objectives of this project are:

1. **Scenario 1 - Human User:** Analyze the system under continuous Poisson arrivals using queueing theory and discrete-event simulation. Determine the minimum queue capacities (K_C , K_G) required to ensure system stability (ergodicity) under different routing policies.
2. **Scenario 2 - Agent:** Formulate and solve an Integer Linear Programming (ILP) model for batch scheduling optimization. Compare optimized scheduling against random assignment to quantify the benefits of optimization.
3. **Comparative Analysis:** Compare the two scenarios and discuss the trade-offs between simulation-based and optimization-based approaches for SLM system management.

2 Scenario 1: Human User

2.1 Problem Description

The system models the processing of user prompts through a multi-stage architecture. Prompts arrive according to a Poisson process with rate $\gamma_P = 5$ prompts/min at CPU 1. After initial processing with service rate $\mu_C = 8$ prompts/min, prompts are routed to one of two GPUs for inference, each with service rate $\mu_G = 5$ prompts/min. Finally, the output is processed on CPU 2 with service rate $\mu_C = 8$ prompts/min.

A feedback mechanism models user satisfaction: the output is accepted with probability $p_A = 0.7$. In the remaining 30% of cases, the job is rejected and re-enters the system at CPU 1, increasing the effective arrival rate. The goal is to find the minimum queue capacities K_C (for CPU2) and K_G (for each GPU) that ensure system stability.

2.2 Queueing Model

The network consists of a hybrid structure. CPU 1 is modeled as an M/M/1 queue with infinite capacity, while the GPUs and CPU 2 are M/M/1/K nodes with finite capacities K_G and K_C , respectively.

Since finite queues saturate rather than diverge, the ergodicity of the entire system depends on the stability of the infinite buffer at CPU 1. If K_G and K_C are insufficient, blocking phenomena occur. For example, a job cannot leave CPU 1 if the target GPU queue is full. This blocking effectively reduces the service rate of CPU 1 and causes its queue to grow indefinitely.

The effective arrival rate at CPU 1 can be computed considering the feedback loop:

$$\lambda_{eff} = \frac{\gamma_P}{p_A} = \frac{5}{0.7} \approx 7.14 \text{ prompts/min} \quad (1)$$

For stability, we require the system to process jobs faster than they arrive at each stage. The simulation aims to find the minimum finite capacities that prevent back-pressure from causing the CPU 1 queue to become unstable.

2.3 Ergodicity Criterion

To determine whether a given configuration (K_C, K_G) results in an ergodic system, we define the following criterion based on average queue lengths observed during steady-state simulation:

- $\bar{X}_{CPU1} < 50$: The average queue length at CPU 1 must remain bounded
- $\bar{X}_{GPU} < 0.8 \times K_G$: Average GPU queue length stays below 80% of capacity
- $\bar{X}_{CPU2} < 0.8 \times K_C$: Average CPU 2 queue length stays below 80% of capacity

A configuration is considered ergodic only if all three conditions are satisfied. We test each (K_C, K_G) combination with 5 different random seeds and require at least 80% (4 out of 5) of the seeds to satisfy the ergodicity criterion. This approach provides statistical confidence in the results.

2.4 Routing Policies

Three different routing strategies were implemented to assign prompts from CPU 1 to the GPUs. The corresponding Simulink models are shown below.

2.4.1 Uniform Probability (50/50)

In this baseline configuration, prompts are routed to GPU 1 or GPU 2 with equal probability ($p = 0.5$), irrespective of the current queue status. If the chosen GPU is full (blocking), the prompt remains in CPU 1 until space becomes available. This blind assignment can lead to inefficiencies where one GPU is idle while the other is congested.

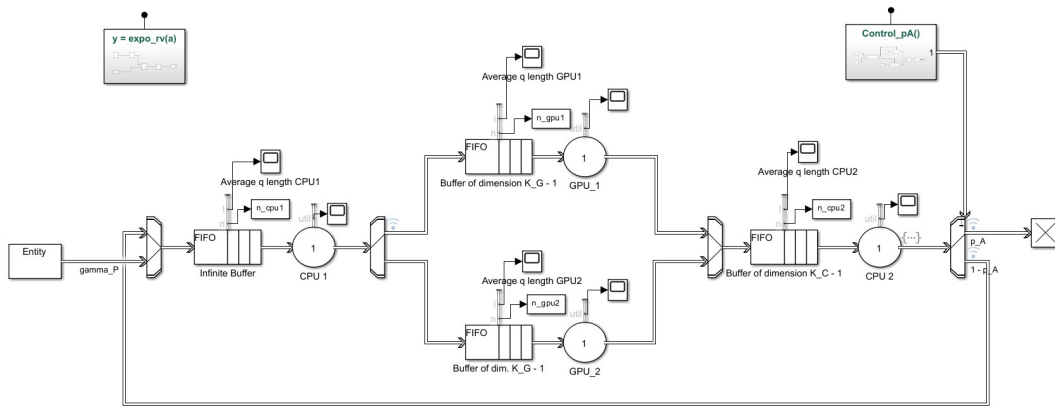


Figure 1: Simulink Model: Uniform Probability Routing

2.4.2 Lower Load Queue (Join Shortest Queue)

This dynamic policy assigns the incoming prompt to the GPU currently holding the fewest items. If both queues have the same load, one is selected at random. By actively balancing the workload, this policy minimizes the probability of blocking and optimizes buffer usage.

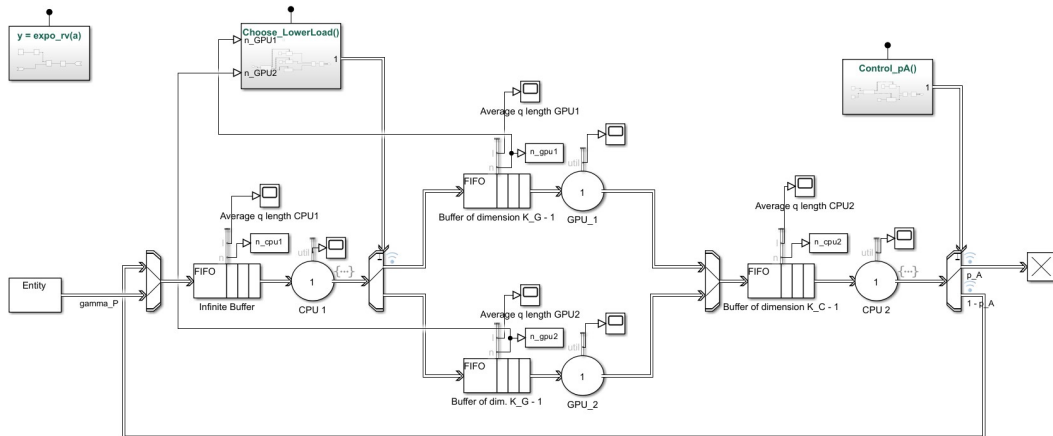


Figure 2: Simulink Model: Lower Load Routing

2.4.3 Unified GPUs (M/M/2/K)

In this scenario, the two GPUs are modeled as a single pooled resource with a shared queue of capacity $2K_G$. A prompt waits in the common queue and is served by the first available GPU. This represents the theoretical optimum for resource utilization, as no server remains idle if there is a pending request.

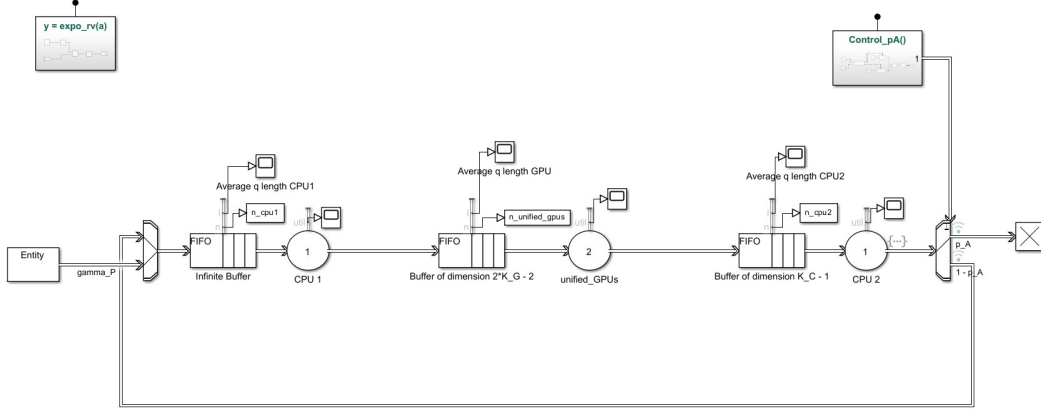


Figure 3: Simulink Model: Unified GPUs (M/M/2/K)

2.5 Simulation Setup

To ensure the robustness and accuracy of the results, we adopted a hierarchical validation strategy consisting of three stages: coarse-grained exploration, fine-grained refinement, and long-run validation.

Stage 1: Exploratory Phase (Broad Sweep)

Initially, an extensive sweep was performed with a short horizon ($T_{sim} = 7000$) to test a wide grid of discrete capacities $K_C, K_G \in \{2, 5, 10, 20, 50, 100, 300\}$. This analysis, comprising over 100 simulation runs across different seeds, allowed us to identify the rough stability boundaries where the system transitions from unstable to ergodic.

Table 1 presents a subset of these initial configurations. The results highlight how insufficient capacity (e.g., $K_C = 2$) leads to instability regardless of K_G , while sufficiently large buffers guarantee stability.

Table 1: Sample of Extended Parameter Sweep Results (Exploratory Phase)

K_C	K_G	$\bar{X}_{CPU1}$	Status	Observation
2	20	311.85	Unstable	Severe blocking at CPU2
5	5	565.17	Unstable	Insufficient total capacity
20	5	160.40	Unstable	GPU bottleneck (K_G too low)
10	10	22.92	Stable	Balanced configuration
20	10	6.10	Stable	Safety margin achieved
50	50	7.47	Stable	Oversized buffers

Stage 2: Refinement Phase

Based on the boundaries identified in the exploratory phase, we narrowed the search space to a denser grid of values to pinpoint the exact minimum capacities. Specifically, we focused on the range $K \in [5, 15]$. For each routing policy, we tested all 25 combinations of these specific (K_C, K_G) values using 5 different random seeds to average out stochastic variance (125 runs per policy).

The specific parameters used for this refinement phase are detailed in Table 2.

Table 2: Simulation Parameters for Refinement Phase

Parameter	Value
Simulation time (T_{sim})	7000 time units
Arrival rate (γ_P)	5 prompts/min
CPU service rate (μ_C)	8 prompts/min
GPU service rate (μ_G)	5 prompts/min
Acceptance probability (p_A)	0.7
K_C values tested	5, 7, 8, 10, 15
K_G values tested	5, 7, 8, 10, 15
Random seeds	1, 2, 3, 4, 5

Stage 3: Confirmatory Phase (Long-Run)

Finally, for the minimum ergodic capacities identified in Stage 2, a confirmatory phase with a significantly longer horizon ($T_{sim} = 300,000$) was executed. This step was crucial to verify that the queue lengths fluctuate around a stable mean without exhibiting slow divergence, confirming the ergodicity of the selected design points. A graphical analysis of the queue evolution during this long-run simulation is presented in the Results section.

2.6 Results

2.6.1 Stability Analysis and Minimum Capacities

The simulations identified the critical capacity regions for each policy. Table 3 summarizes the minimum queue capacities required for ergodicity. The values were determined through the exploratory sweep and refined in the second simulation phase.

Table 3: Minimum Queue Capacities for Ergodicity

Routing Policy	K_C (min)	K_G (min)	Total	Notes
Uniform Probability (50/50)	8	10	18	Highest capacity needed
Lower Load Queue	5	7	12	33% reduction
Unified GPUs (M/M/2/K)	5	7	12	Theoretical Optimum

To visualize the stability boundaries, Figure 4 presents the ergodicity heatmaps. Green cells indicate ergodic configurations, while red cells indicate instability. It is evident that the "Lower Load" policy expands the stability region compared to the Uniform case, allowing for tighter buffer constraints. This advantage is further highlighted in Figure 5,

which directly compares the resource requirements: the adoption of smart routing leads to a significant saving in memory resources compared to the naive approach.

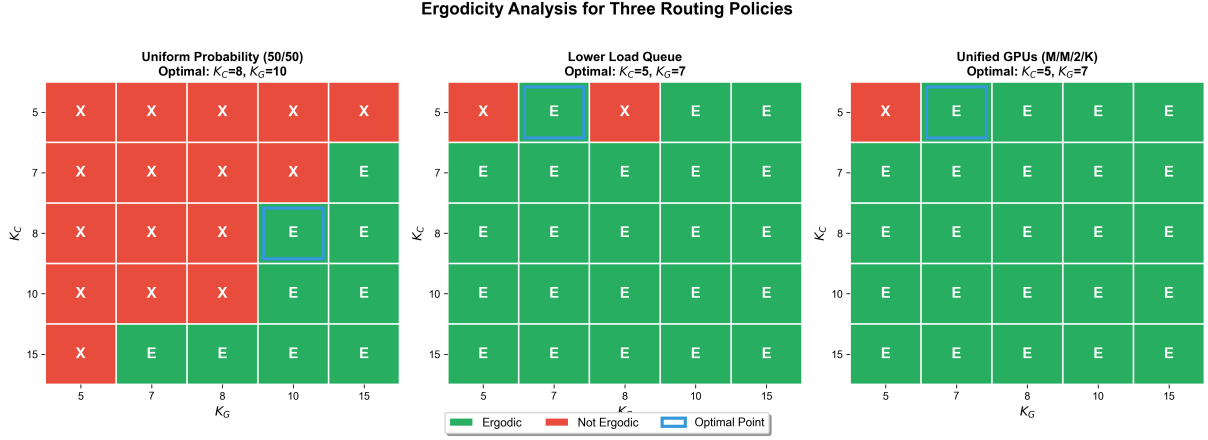


Figure 4: Ergodicity Heatmaps for Three Routing Policies. Left: Uniform Probability requires larger buffers ($K_C \geq 8, K_G \geq 10$). Center and Right: Smart routing policies achieve ergodicity with reduced capacities ($K_C \geq 5, K_G \geq 7$).

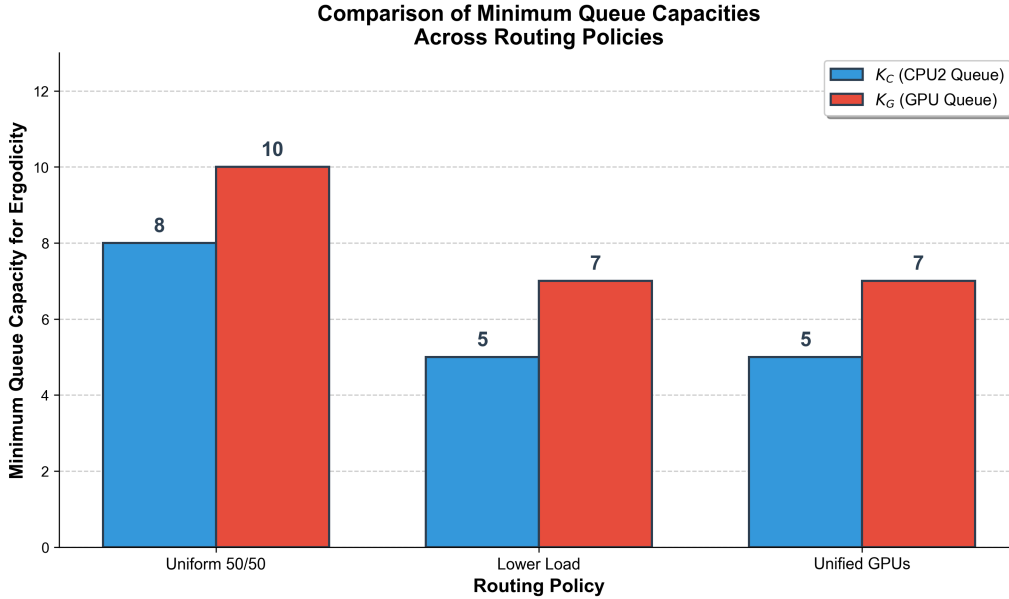


Figure 5: Comparison of Minimum Queue Capacities. The Uniform Probability policy requires significantly larger buffers than the smart routing policies.

2.6.2 Validation of the Optimal Policy (Lower Load)

To validate the robustness of the recommended configuration (*Lower Load Queue*), we analyzed the system dynamics over a long-run simulation ($T_{sim} = 300,000$).

Figure 6 confirms system stability, showing that the CPU 1 average queue length fluctuates around a constant mean without diverging. Furthermore, Figure 7 illustrates the efficiency of the load balancing. The utilization plots show that the workload is evenly distributed between the two GPUs, maximizing the effective capacity of the parallel stage.

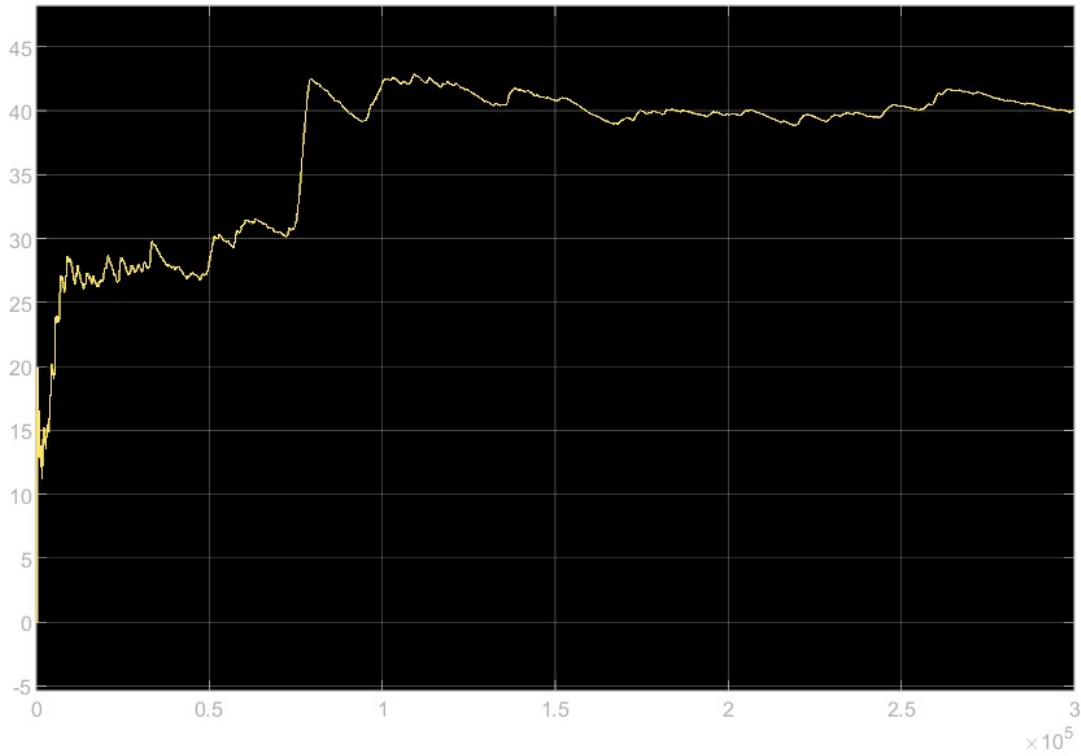


Figure 6: Trace of CPU 1 Average Queue Length (Lower Load Policy, $T = 300,000$). The stationary behavior confirms system stability.

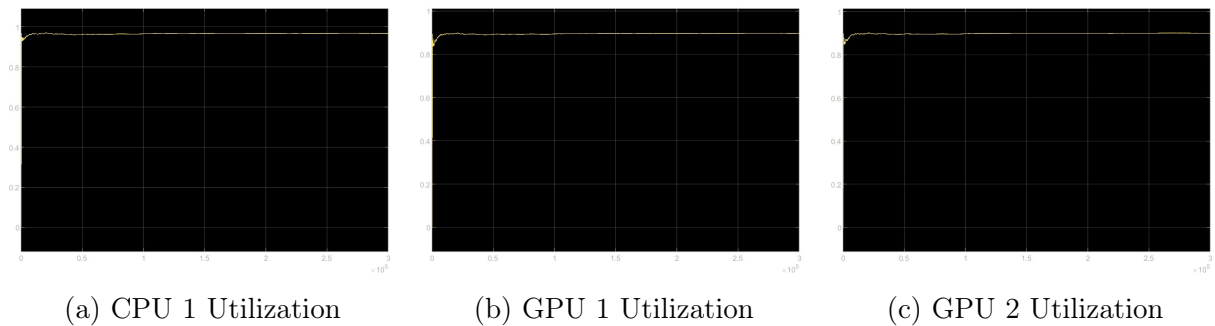


Figure 7: Server Utilizations under Lower Load Policy. The workload is perfectly balanced between GPU 1 and GPU 2 ($\rho_{G1} \approx \rho_{G2}$), maximizing parallel efficiency.

2.7 Analysis

The results clearly demonstrate the impact of the routing strategy on hardware sizing:

1. **Uniform Probability (50/50):** This naive approach requires the largest buffers ($K_C = 8$, $K_G = 10$). Random assignment leads to high variance in queue lengths; it is common for one GPU to be idle while the other overflows, causing blocking at CPU 1.
2. **Lower Load Queue:** By routing jobs to the GPU with fewer waiting items, this policy achieves a **33% reduction** in total buffer requirements ($K_C = 5$, $K_G = 7$). As shown in Fig. 7, this policy equalizes the utilization of the parallel resources.

3. **Unified GPUs (M/M/2/K):** This configuration achieves the same efficiency as the Lower Load Queue ($K_{tot} = 12$), representing the theoretical optimum for parallel server utilization. It is worth noting that $K_G > K_C$ (7 vs 5) across all stable configurations because the GPUs have a slower service rate ($\mu_G = 5$) compared to CPU 2 ($\mu_C = 8$), making them more sensitive to burst arrivals.

2.7.1 Why Smart Routing Works

The Lower Load Queue and Unified GPUs policies outperform random routing because they minimize the *coefficient of variation* of the queue lengths across the GPUs. In a random policy, there is a non-zero probability that $Queue_{G1} = K_G$ (full) while $Queue_{G2} = 0$ (empty). In this state, system capacity is wasted and blocking occurs. Smart policies prevent this state, effectively ensuring that if there is capacity available, it is used. This reduces the maximum queue length observed at any single GPU, thereby reducing the required buffer size K_G .

2.7.2 Practical Recommendation

For real SLM deployments, the **Lower Load Queue** policy is recommended as the best trade-off. It achieves the same buffer efficiency as the theoretical Unified model ($K_{tot} = 12$) but is easier to implement in distributed systems, as it only requires monitoring queue lengths rather than maintaining a complex shared memory architecture.

3 Scenario 2: Agent

3.1 Problem Description

In the agent scenario, a batch of N prompts arrives simultaneously and must be processed through the SLM system. Unlike the human user scenario with continuous Poisson arrivals, all jobs are known in advance, which enables optimization of the processing schedule.

The system processes each prompt through three stages:

1. **CPU1 (Pre-processing):** Tokenization with service rate $\mu_C = 8$ prompts/min
2. **GPU1 or GPU2 (Inference):** Parallel processing with service rate $\mu_G = 5$ prompts/min each
3. **CPU2 (Post-processing):** De-tokenization with service rate $\mu_C = 8$ prompts/min

Processing times are exponentially distributed with means $1/\mu_C = 0.125$ min for CPU stages and $1/\mu_G = 0.2$ min for GPU stages.

Objective: Minimize the makespan C_{max} , defined as the completion time of the last job leaving the system.

3.2 Mathematical Formulation

We formulate the problem as an Integer Linear Program (ILP) using the Big-M method for disjunctive constraints.

3.2.1 Notation

- N : Number of jobs (prompts) in the batch
- $P_{m,j}$: Processing time of job j on machine m
- Machines: $m \in \{1, 2, 3, 4\}$ representing CPU1, GPU1, GPU2, CPU2
- M : Large constant for Big-M method ($M = 1000$)

3.2.2 Decision Variables

- $C_{m,j} \geq 0$: Completion time of job j on machine m (continuous)
- $x_j \in \{0, 1\}$: GPU assignment variable
 - $x_j = 1$: Job j is processed on GPU1
 - $x_j = 0$: Job j is processed on GPU2
- $y_{i,j} \in \{0, 1\}$: Sequencing variable for jobs i and j
 - $y_{i,j} = 1$: Job i precedes job j on shared machines
 - $y_{i,j} = 0$: Job j precedes job i
- $C_{max} \geq 0$: Makespan (continuous)

3.2.3 Objective Function

$$\min \quad C_{max} \quad (2)$$

3.2.4 Constraints

1. Makespan definition:

$$C_{max} \geq C_{4,j} \quad \forall j \in \{1, \dots, N\} \quad (3)$$

2. Release time constraint (jobs start at time 0):

$$C_{1,j} \geq P_{1,j} \quad \forall j \in \{1, \dots, N\} \quad (4)$$

3. Flow constraints (precedence between stages):

If job j uses GPU1 ($x_j = 1$):

$$C_{2,j} \geq C_{1,j} + P_{2,j} - M(1 - x_j) \quad (5)$$

$$C_{4,j} \geq C_{2,j} + P_{4,j} - M(1 - x_j) \quad (6)$$

If job j uses GPU2 ($x_j = 0$):

$$C_{3,j} \geq C_{1,j} + P_{3,j} - M \cdot x_j \quad (7)$$

$$C_{4,j} \geq C_{3,j} + P_{4,j} - M \cdot x_j \quad (8)$$

4. Disjunctive constraints (no overlap on same machine):

For each machine, we ensure that jobs do not overlap. For CPU1 and CPU2 (shared by all jobs):

$$C_{m,j} - P_{m,j} \geq C_{m,i} - M(1 - y_{i,j}) \quad \forall i < j \quad (9)$$

$$C_{m,i} - P_{m,i} \geq C_{m,j} - M \cdot y_{i,j} \quad \forall i < j \quad (10)$$

For GPUs, similar constraints apply only when both jobs use the same GPU, with additional Big-M terms for the x_j variables.

3.3 Solution Approaches

We compare two approaches to evaluate the benefit of optimization:

3.3.1 Variant 1: Random Order Simulation

Jobs are processed in arrival order (1, 2, ..., N) with random GPU assignment. Each job is independently assigned to GPU1 or GPU2 with equal probability (0.5). This represents a naive baseline without optimization.

3.3.2 Variant 2: ILP Optimization

The ILP model simultaneously optimizes:

- Job sequence on each machine (via $y_{i,j}$ variables)
- GPU assignment for each job (via x_j variables)

The problem is solved using MATLAB's `intlinprog` solver with a 60-second time limit per instance.

3.4 Experimental Setup

Table 4: Experimental Parameters

Parameter	Value
Batch sizes (N)	5, 7, 9
Random seeds	1, 2, 3, 4, 5
CPU service rate (μ_C)	8 prompts/min
GPU service rate (μ_G)	5 prompts/min
Big-M constant	1000
Solver time limit	60 seconds

For each combination of batch size and seed, we:

1. Generate processing times using the exponential distribution
2. Compute makespan using random simulation
3. Solve the ILP to find optimal makespan
4. Compare results and calculate improvement percentage

3.5 Results

3.5.1 Summary by Batch Size

Table 5 shows the average performance across all seeds for each batch size.

Table 5: Average Results by Batch Size

Batch Size (N)	Avg Random	Avg Optimized	Improvement
5	1.6981	1.1666	31.30%
7	2.1588	1.4195	34.24%
9	2.1897	1.7270	21.13%

The ILP optimization outperforms random scheduling in all 15 test instances, with individual improvements ranging from 10% to 59%.

3.6 Visualization

3.6.1 Gantt Charts

Figure 8 shows Gantt charts comparing random and optimized schedules for $N = 7$. Each colored bar represents a job execution on each machine. The red dashed line indicates the makespan.

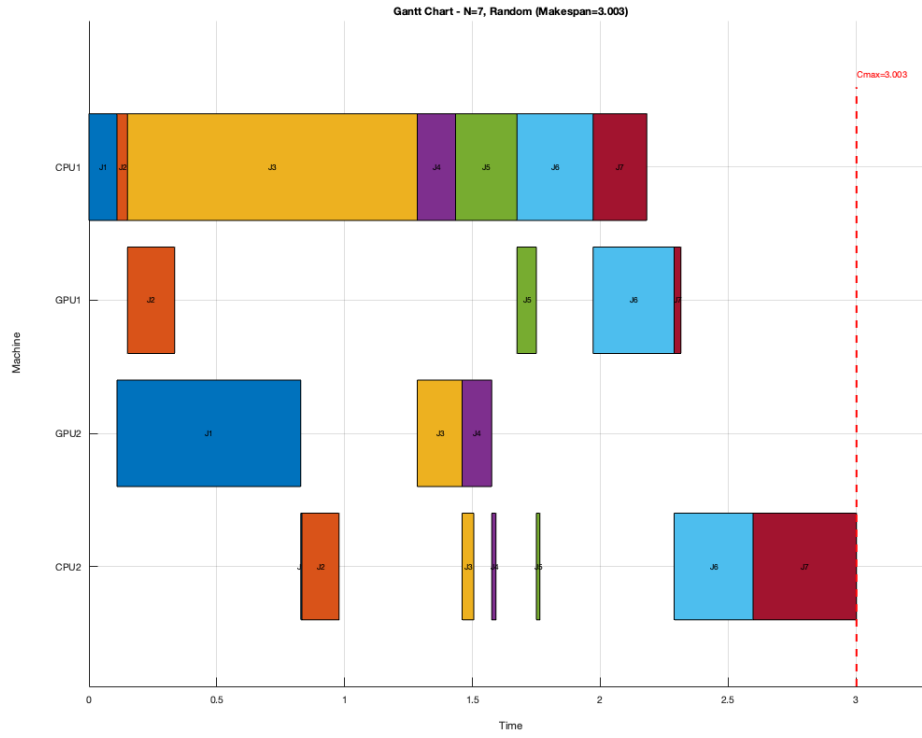
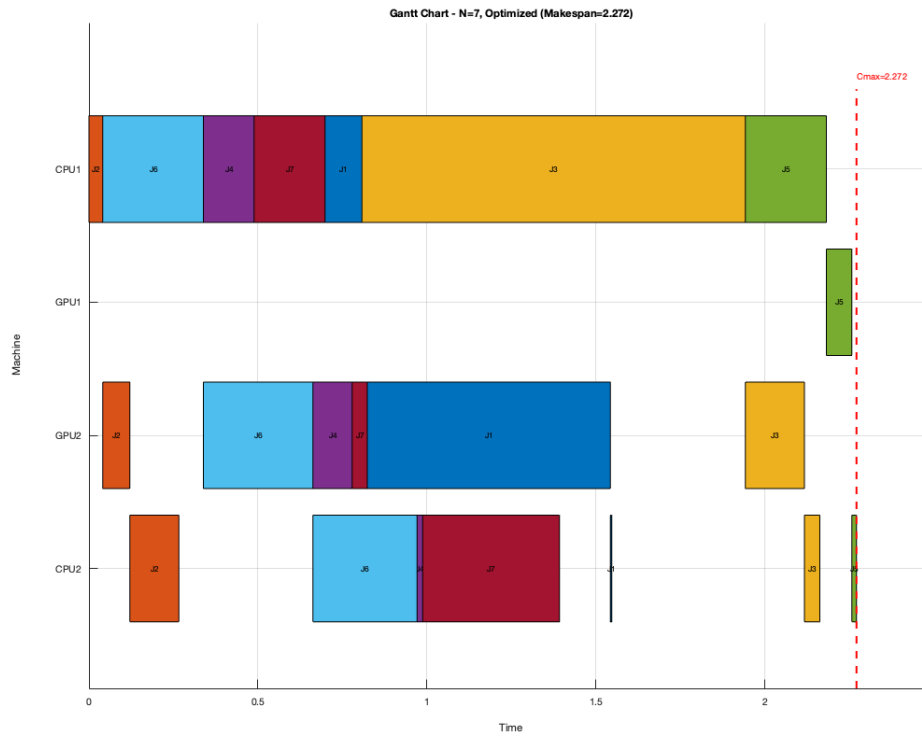
(a) Random Scheduling ($C_{max} = 3.003$)(b) Optimized Scheduling ($C_{max} = 2.272$)

Figure 8: Gantt Charts for N=7 (Seed=1): The optimized schedule achieves 24.35% improvement through better GPU load balancing and job sequencing.

3.6.2 Performance Comparison

Figure 9 shows the average makespan comparison across all batch sizes.

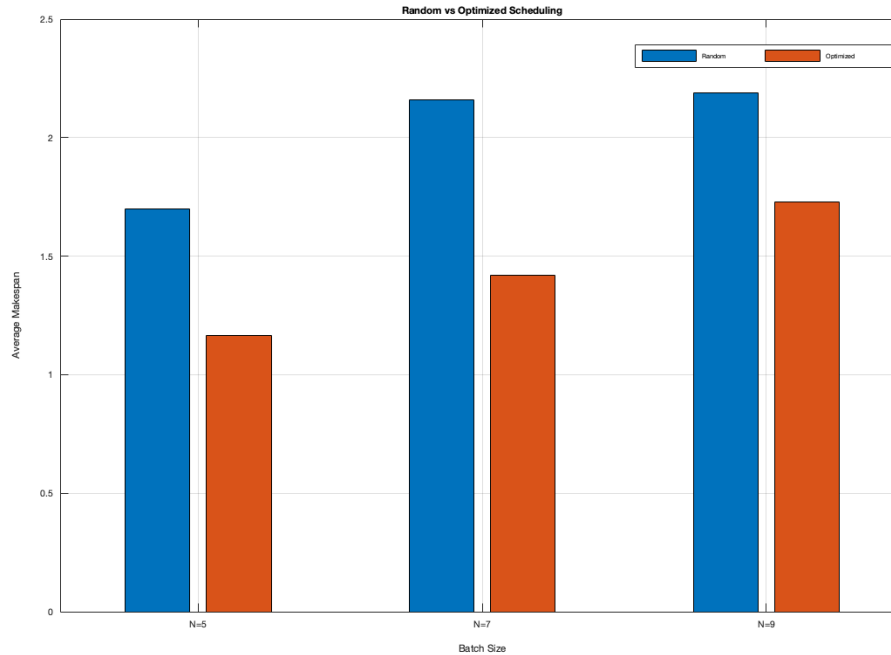


Figure 9: Average Makespan Comparison: Random vs Optimized Scheduling.

3.7 Analysis

The optimization achieves makespan reduction through:

- **Intelligent GPU Assignment:** The optimizer balances workload between GPU1 and GPU2 based on actual processing times.
- **Optimal Job Sequencing:** Jobs are ordered to minimize idle time between consecutive operations.
- **Reduced Machine Idle Time:** The Gantt charts show that optimized schedules have less idle time compared to random schedules.

3.7.1 Why Medium Batch Sizes Show Best Improvement

The results show that $N = 7$ achieves the highest improvement (34.24%), while $N = 5$ shows 31.30% and $N = 9$ shows only 21.13%. This pattern occurs because:

- **Small batches ($N = 5$):** Limited optimization potential, with few jobs, even random assignment often achieves reasonable GPU balance by chance.
- **Medium batches ($N = 7$):** Optimal trade-off, enough jobs to create significant imbalance under random assignment, but the ILP can still find the optimal solution quickly.
- **Larger batches ($N = 9$):** The random baseline becomes more predictable (law of large numbers helps balance loads), reducing the relative advantage of optimization.

3.7.2 Sources of Makespan Reduction

Analyzing the Gantt charts reveals three key optimization strategies:

1. **GPU Load Balancing:** The optimizer assigns jobs such that total processing time on GPU1 $\approx$ total on GPU2. Random assignment may put 5 jobs on one GPU and 2 on the other.
2. **Short-Job-First on Shared Resources:** Jobs with shorter CPU processing times tend to be scheduled earlier, reducing blocking for subsequent jobs.
3. **Pipeline Optimization:** The optimizer sequences jobs to minimize gaps between CPU1 completion and GPU start, and between GPU completion and CPU2 start.

3.7.3 Practical Implications

For real SLM batch processing:

- **Optimization is worth the effort:** A 21-34% reduction in processing time translates directly to faster response for batch requests (e.g., agents running multiple queries).
- **Solution time is acceptable:** All instances solved optimally within seconds, making ILP practical for real-time decision making when $N \leq 10$.
- **For larger batches:** When $N > 15$, consider heuristics like Longest Processing Time (LPT) for GPU assignment, which provides near-optimal results in $O(N \log N)$ time.

4 Results Comparison

This section compares the findings from both scenarios and discusses their practical implications for SLM system design and operation.

4.1 Summary of Findings

4.1.1 Scenario 1: Human User

The simulation study revealed significant differences between routing policies:

- **Uniform Probability (50/50):** Minimum $K_C = 8$, $K_G = 10$ for ergodicity (total = 18)
- **Lower Load Queue:** Minimum $K_C = 5$, $K_G = 7$ for ergodicity (total = 12)
- **Unified GPUs (M/M/2/K):** Minimum $K_C = 5$, $K_G = 7$ for ergodicity (total = 12)

Smart routing policies (Lower Load and Unified GPUs) reduce the total buffer requirement by 33% compared to naive random routing. This finding emphasizes the importance of intelligent load balancing in queueing systems with parallel servers.

4.1.2 Scenario 2: Agent

The optimization study demonstrated consistent improvements over random scheduling:

- Average improvement of 21–34% in makespan compared to random scheduling
- Best performance at $N = 7$ with 34.24% average improvement
- Consistent benefits across all 15 test instances (improvement range: 10% to 59%)
- Effective load balancing by intelligently distributing jobs between GPU1 and GPU2

4.2 Comparison of Approaches

Table 6 summarizes the key differences between the two scenarios.

Table 6: Comparison of Two Scenarios

Aspect	Scenario 1 (Human User)	Scenario 2 (Agent)
Arrival Pattern	Continuous (Poisson)	Batch (N jobs at once)
Information	Jobs unknown in advance	All jobs known
Method	Discrete-event simulation	Integer Linear Programming
Tool	MATLAB Simulink	MATLAB Optimization Toolbox
Objective	System stability	Minimize makespan
Decision Variables	Queue capacities (K_C, K_G)	Job sequence & GPU assignment
Key Metric	Ergodicity condition	C_{max} (completion time)
Time Horizon	Long-term (steady state)	Short-term (single batch)

4.3 Complementary Nature of Approaches

The two scenarios address different aspects of SLM system management:

- **Scenario 1** answers: “How much buffer capacity do we need to handle random arrivals without losing requests?”
- **Scenario 2** answers: “Given a batch of requests, what is the fastest way to process them all?”

Together, they provide a comprehensive view of system performance under different operational conditions.

4.4 Practical Implications

4.4.1 When to Use Each Approach

Scenario 1 (Simulation) is appropriate for continuous arrivals from multiple users, steady-state operation, and capacity planning where the goal is to ensure stability.

Scenario 2 (Optimization) is appropriate when jobs are known in advance, computation time for optimization is acceptable, and the goal is to minimize total processing time.

4.5 Key Insights

1. **GPU is the bottleneck:** With $\mu_G = 5 < \mu_C = 8$, the inference stage limits system throughput. Having two parallel GPUs helps mitigate this.
2. **Parallel resources enable optimization:** The presence of two GPUs creates assignment decisions that optimization can exploit for significant improvements.
3. **Trade-off between simplicity and performance:** Random assignment is simple but suboptimal; intelligent routing/optimization requires more computation but yields 21-34% better performance.

4.6 Connecting the Two Scenarios

Although the scenarios appear different (continuous vs. batch arrivals), they share a fundamental insight: **intelligent resource allocation matters**.

- In Scenario 1, “intelligence” means routing jobs to the less loaded GPU in real-time
- In Scenario 2, “intelligence” means optimizing the entire schedule before execution

Both achieve approximately **30% improvement** over naive approaches (33% buffer reduction in Scenario 1, 21-34% makespan reduction in Scenario 2). This consistency suggests that intelligent resource management provides roughly one-third improvement in systems with parallel processing stages.

4.7 System Design Recommendations

Based on our analysis, we recommend the following for SLM system deployment:

1. **For Human-Facing Applications:**
 - Implement Lower Load Queue routing between GPUs
 - Size buffers at $K_C = 5$, $K_G = 7$ per GPU (or 80% of these values with acceptance of occasional blocking)
 - Monitor GPU queue lengths to detect capacity issues early
2. **For Agent/Batch Applications:**
 - Use ILP optimization for small batches ($N \leq 10$) where solution time is negligible
 - For larger batches, use LPT heuristic: assign jobs to GPUs in order of decreasing processing time, always choosing the GPU with less total assigned work
 - Pre-compute processing time estimates based on prompt length/complexity
3. **For Mixed Workloads:**
 - Prioritize human requests (latency-sensitive) over agent batches (throughput-sensitive)
 - Use dynamic buffer sizing: increase K_G during high-load periods
 - Consider dedicating one GPU to human requests and one to agent batches during peak times

5 Conclusion

This project analyzed a Small Language Model (SLM) system under two operational scenarios: continuous arrivals (Human User) and batch processing (Agent). Using discrete-event simulation and integer linear programming, we demonstrated effective approaches for system capacity planning and schedule optimization.

Scenario 1: The queueing analysis revealed that smart routing policies significantly impact system stability. The Uniform Probability policy requires $K_C = 8$ and $K_G = 10$ (total capacity = 18) for ergodicity, while both Lower Load Queue and Unified GPUs achieve stability with only $K_C = 5$ and $K_G = 7$ (total capacity = 12), a 33% reduction. The GPU stage is the bottleneck due to its slower service rate ($\mu_G = 5 < \mu_C = 8$), making proper load balancing essential.

Scenario 2: The ILP optimization consistently outperforms random scheduling, with improvements ranging from 10% to 59% across all test instances. Average improvements are 31% for $N = 5$, 34% for $N = 7$, and 21% for $N = 9$. All instances were solved optimally within 60 seconds.

In summary, this project successfully applied both simulation and optimization techniques. For continuous arrivals, smart routing reduces buffer requirements by 33%. For batch processing, ILP optimization reduces makespan by 21-34% on average.